

ARYABHATTA KNOWLEDGE UNIVERSITY

PATNA, BIHAR

BCA-304

SOFTWARE ENGINEERING

COMPLETE EXAM-COPY STUDY NOTES

BCA Semester III | Full Marks: 60 | Time: 3 Hours

PYQ Years Covered: 2014 · 2017 · 2018 · 2019 · 2020 · 2022 · 2024 · 2025

UNIT	TOPIC	KEY AREAS
1	Introduction to SE + Life Cycle Models	Waterfall · Prototype · Spiral · Metrics
2	Software Project Management + SRS	LOC · FP · COCOMO · Delphi · SRS Document
3	Software Design	DFD · Structure Charts · FOD · OOD
4	Coding and Testing + Reliability	Unit · Black-Box · White-Box · Musa Model
5	Software Quality Assurance + Maintenance	ISO 9000 · SEI CMM · Maintenance Types

★ MOST REPEATED TOPICS: Waterfall Model (8×) · Software Maintenance (8×) · ISO 9000 vs SEI CMM (7×) · Black/White Box Testing (6×) · SRS Document (6×) · COCOMO Model (5×) · DFD (5×) · Spiral Model (5×)

FORMAT: Section A = MCQ + Answers | Section B = 4-Mark Q&A | Section C = 12-Mark Long Answers

UNIT 1 — INTRODUCTION TO SOFTWARE ENGINEERING + LIFE CYCLE MODELS

Characteristics | Software Crisis | Process & Product Metrics | Waterfall | Prototype | Spiral Models

SECTION A — MCQ WITH ANSWERS (2 Marks Each)

Q1. What is the main concern of Software Engineering area?

■ PYQ: 2014

- (A) Hardware configuration
- ✓ (B) **Software Production** ← CORRECT ANSWER
- (C) Network Configuration
- (D) Software Reusability

Explanation: Software Engineering focuses on the systematic production of high-quality software — not hardware or networking.

Q2. The WORST type of coupling is:

■ PYQ: 2014, 2019

- (A) Data coupling
- (B) Control coupling
- (C) Stamp coupling
- ✓ (D) **Content coupling** ← CORRECT ANSWER

Explanation: Content coupling is worst — one module directly accesses internal data of another, violating information hiding.

Q3. Requirements can be refined using which model?

■ PYQ: 2014

- (A) Waterfall model
- ✓ (B) **Prototyping model** ← CORRECT ANSWER
- (C) Evolutionary model
- (D) Spiral model

Explanation: Prototype model lets users see and interact with a working model, making requirement refinement easy and clear.

Q4. The most important feature of the Spiral Model is:

■ PYQ: 2019

- (A) Efficiency management
- (B) Quality management
- ✓ (C) **Risk management** ← CORRECT ANSWER
- (D) Performance management

Explanation: Spiral model is risk-driven — every cycle starts with risk analysis and mitigation. Risk management is its core.

Q5. Waterfall model is NOT suitable for:

■ PYQ: 2019

- (A) Small projects
- (B) Complex projects
- ✓ (C) **Accommodating changes** ← CORRECT ANSWER
- (D) None of above

Explanation: Waterfall is rigid and sequential. Once a phase is complete, going back is very expensive — not good for changes.

Q6. SDLC stands for:

■ PYQ: 2018

- (A) Software Design Life Cycle
- (B) Software Development Life Cycle
- (C) System Development Life Cycle

✓ (D) Both B and C ← CORRECT ANSWER

Explanation: SDLC = Software Development Life Cycle OR System Development Life Cycle — both expansions are commonly used.

Q7. What is the tool that supports different stages of Software Development Life Cycle?

■ PYQ: 2014

✓ (A) CASE tool ← CORRECT ANSWER

- (B) CAME tool
- (C) CAQE tool
- (D) CARE tool

Explanation: CASE = Computer Aided Software Engineering tools support all SDLC phases from analysis to maintenance.

Q8. The life cycle model that combines Waterfall and Prototyping with risk analysis is:

■ PYQ: 2017, 2022

- (A) Incremental model
- (B) RAD model

✓ (C) Spiral model ← CORRECT ANSWER

- (D) Agile model

Explanation: Spiral model = Waterfall (structured) + Prototyping (iterative) + Risk Analysis at each cycle.

SECTION B — 4-MARK SHORT ANSWERS

Q1. What is the Prototype Model? When is it beneficial to construct a Prototype?

4 Marks

■ PYQ: 2014, 2022

📌 ANSWER:

Definition:

Prototype Model is an iterative approach where a preliminary working version of the software is built quickly to gather user feedback before full development begins.

When is it Beneficial?

1. When **requirements are unclear or incomplete** and users cannot define them precisely.
2. When **new technology** is being used and feasibility is uncertain.
3. When the system needs a **high level of user interaction** (like UI-heavy apps).
4. When **high-risk elements** need to be validated before full development.

Prototype Process:

Gather basic requirements → Build prototype quickly → Show to user → Get feedback → Refine → Repeat → Final system

Advantage: Reduces misunderstanding between developer and user. Finds errors early.

Disadvantage: Can lead to scope creep; poor structure if prototype code reused.

Q2. List the important shortcomings of LOC (Lines of Code) as a software size metric.

4 Marks

■ PYQ: 2014

👉 ANSWER:

Shortcomings of LOC:

1. **Language dependent** — Same program needs more lines in C++ than in Python. Makes comparison unfair.
2. **Available only after coding** — Cannot estimate project size early in planning phase.
3. **Penalizes efficient programmers** — A skilled programmer writing 100 lines may do more than someone writing 1000 lines.
4. **Programmer style varies** — Some write compact code, some use verbose style. No consistency.
5. **Does not reflect functionality** — More lines of code does NOT mean more useful software.
6. **Counting method varies** — Are blank lines and comments counted? Different teams count differently.

Alternative: Function Points (FP) overcome these limitations — language-independent, available early, measures user functionality.

Q3. What is the Software Crisis? How did it lead to Software Engineering?

4 Marks

■ PYQ: 2017, 2022

👉 ANSWER:

Software Crisis (1960s–1970s):

Software Crisis was a set of serious problems affecting the software industry in the 1960s and 70s.

Problems of Software Crisis:

1. Projects were **delivered very late** — months or years behind schedule.
2. Projects **exceeded budgets** — sometimes by 100% or more.
3. Software was **unreliable** — full of bugs, crashed frequently.
4. Code was **very hard to maintain** — poor documentation, no structure.
5. Software did **not meet user needs** — wrong system built.

How SE Solved It:

1. Introduced **systematic, disciplined development** process (SDLC phases).
2. Defined formal **lifecycle models** — Waterfall, Prototype, Spiral.
3. IEEE defined SE as: "*Systematic, disciplined, quantifiable approach to software development.*"
4. Brought **engineering principles** — documentation, testing, quality assurance.

Q4. What are Process Metrics and Product Metrics? Give examples.

4 Marks

■ PYQ: 2018, 2020

👉 ANSWER:

Process Metrics:

Process Metrics measure the efficiency and effectiveness of the **software development process** itself.

1. **Defect Removal Efficiency** = Defects found before delivery ÷ Total defects × 100%
2. **Effort Variance** = Actual effort – Estimated effort
3. **Schedule Variance** = Actual time – Planned time

Product Metrics:

Product Metrics measure the characteristics and quality of the **software product** itself.

1. **Size Metrics** — Lines of Code (LOC), Function Points (FP)
2. **Complexity Metrics** — Cyclomatic complexity, nesting depth
3. **Quality Metrics** — Reliability, maintainability, usability scores

Key Rule: "You cannot control what you cannot measure." Metrics are essential for project management.

SECTION C — 12-MARK LONG ANSWERS

Q1.

Explain the Waterfall Model with diagram. What are its advantages and disadvantages? When is it suitable?

12 Marks

■ PYQ: 2014, 2017, 2018, 2019, 2020, 2022, 2024, 2025

👉 ANSWER:

Introduction:

The **Waterfall Model** is the oldest and most widely known software development model. It follows a **linear sequential approach** where each phase must be completed fully before the next phase begins — like water flowing down a waterfall.

Definition:

Waterfall Model: A linear, sequential software development process where phases flow downward (Requirements → Design → Implementation → Testing → Deployment → Maintenance) with **minimal backward movement**.

Six Phases of Waterfall Model:

1. **Requirements Gathering** — Stakeholders and analysts document ALL user requirements completely and precisely. Output: SRS Document.
2. **System Design** — Architects design the overall system architecture, data structures, interfaces, algorithms, and database schemas. Output: Design Document.
3. **Implementation (Coding)** — Developers write source code based on design specifications. This is the most visible phase. Output: Software modules.
4. **Testing** — Quality assurance team verifies system meets all requirements using unit testing, integration testing, system testing, and UAT. Output: Test Reports.
5. **Deployment** — Completed, tested software is released to end-users in the production environment. May include data migration, user training, system configuration.
6. **Maintenance** — Post-deployment support: bug fixes, performance optimization, minor enhancements. Typically the longest phase (60–80% of total cost).

■ DIAGRAM — WATERFALL MODEL — Sequential Flow

Requirements Gathering ↓ System Design ↓ Implementation (Coding) ↓ Testing (Unit → Integration → System → UAT) ↓ Deployment (Release to Production) ↓ Maintenance (Bug fixes, Updates) Each phase produces a deliverable that feeds the next phase. ← Feedback loop only at great cost.

Advantages of Waterfall Model:

1. **Simple and easy to understand** — linear, logical flow anyone can follow.
2. **Clear documentation** at each phase — serves as knowledge base.
3. **Well-suited for small projects** with stable, well-defined requirements.
4. **Structured and predictable** — easy to manage and track progress.
5. Works well when **technology and domain are well-understood**.

Disadvantages of Waterfall Model:

1. **Rigid and inflexible** — once a phase is complete, going back is very costly.
2. **Late detection of defects** — testing happens near the end; fixing late bugs is expensive.
3. **No working software** until very late in the project — stakeholders cannot see progress.
4. **High risk at end** — if requirements were wrong, the entire project may fail at testing.
5. **Not suitable for complex or evolving** projects where requirements change frequently.

When to Use Waterfall Model:

1. Requirements are **clear, stable, and well-documented** before project starts.
2. Project is **small to medium in size** and complexity.
3. **Technology is well-understood** by the development team.
4. Customer does **not need frequent involvement** during development.

Conclusion:

Waterfall model is simple and disciplined but **too rigid** for modern software projects where requirements change. For projects with unclear or evolving requirements, **Spiral or Agile models** are preferred.

Q2. Explain the Spiral Model with its 4 quadrants. How does it handle risk? Compare Waterfall vs Spiral.

12 Marks

■ PYQ: 2014, 2017, 2019, 2022

📌 ANSWER:

Introduction:

The **Spiral Model**, proposed by Barry Boehm in 1986, is a **risk-driven iterative development model** that combines the systematic nature of Waterfall with the iterative refinement of Prototyping. It explicitly focuses on identifying and mitigating risks at each cycle.

Four Quadrants of Each Spiral Cycle:

1. **Quadrant 1 — Planning (Objectives):** Define objectives, alternatives, constraints for current iteration. Understand what needs to be accomplished.
2. **Quadrant 2 — Risk Analysis:** Identify, analyze, and prioritize risks. Build prototypes to resolve high-risk areas. Create risk mitigation strategies.
3. **Quadrant 3 — Development & Testing:** Develop the product for current iteration. Test to verify correctness. Actual coding and unit testing happen here.
4. **Quadrant 4 — Evaluation & Review:** Customer evaluates the current iteration. Reviews and approves. Plans for the next iteration based on feedback.

■ DIAGRAM — SPIRAL MODEL — Risk-Driven Cycles

PLANNING | RISK ANALYSIS Define objectives | Identify & resolve risks Plan resources | Build prototypes for risks
 EVALUATION & REVIEW | DEVELOPMENT & TESTING Customer reviews output | Code, test, integrate Plan next iteration | Produce working build Each loop of the spiral = one complete project phase Inner loops = early iterations | Outer loops = later iterations

Risk Management in Spiral Model:

1. Risk is **explicitly identified** at the start of EVERY cycle (Quadrant 2).
2. **Prototypes are built** to address unclear or high-risk requirements early.
3. If risk is too high to manage, the **project can be stopped** — saving resources.
4. **Risk resolution** drives development — less risky items done first.

Waterfall vs Spiral Comparison:

WATERFALL vs SPIRAL		
Feature	Waterfall Model	Spiral Model
Approach	Linear, sequential	Iterative, risk-driven cycles
Risk Handling	Low — late detection	High — explicitly at each iteration
Flexibility	Rigid, no changes once phase done	Flexible, changes accommodated each cycle
Customer Involvement	Start and end only	Every iteration (evaluation phase)
Documentation	Very high — formal docs each phase	Moderate — per iteration
Best For	Small, stable, clear requirements	Large, complex, high-risk projects
Cost Predictability	High	Medium — depends on iterations needed

ANSWER:

Conclusion:

Spiral model is ideal for **large, complex, high-risk projects** where requirements evolve and risk management is critical. Its main drawback is that it requires **skilled risk analysts** and can be expensive due to multiple iterations.

UNIT 2 — SOFTWARE PROJECT MANAGEMENT + SOFTWARE REQUIREMENTS SPECIFICATION

Size Estimation (LOC, FP) | Cost Estimation (Delphi, Basic COCOMO) | Halstead | Putnam SLIM | SRS

SECTION A — MCQ WITH ANSWERS (2 Marks Each)

Q1. COCOMO stands for:

■ PYQ: 2014, 2025

- (A) Computer Cost Model
- ✓ (B) **Constructive Cost Model** ← CORRECT ANSWER
- (C) Computed Cost Method
- (D) Constructive Compute Model

Explanation: COCOMO = Constructive Cost Model by Barry Boehm (1981). Based on analysis of 63 real projects.

Q2. In size-oriented metrics, metrics are developed based on:

■ PYQ: 2018

- (A) Number of functions
- (B) Number of user inputs
- ✓ (C) **Number of lines of code** ← CORRECT ANSWER
- (D) Amount of memory usage

Explanation: Size-oriented metric = LOC (Lines of Code) — most basic and direct measure of software size.

Q3. The final outcome of Requirement Analysis and Specification phase is:

■ PYQ: 2018, 2024, 2025

- ✓ (A) **SRS document** ← CORRECT ANSWER
- (B) Source code
- (C) User manual
- (D) Design document

Explanation: SRS (System Requirements Specification) is the formal output of requirements analysis — a contract between users and developers.

Q4. Which is NOT a characteristic of a good SRS document?

■ PYQ: 2019

- (A) Traceable
- (B) Verifiable
- ✓ (C) **Ambiguous** ← CORRECT ANSWER
- (D) Concise

Explanation: Good SRS must be UNAMBIGUOUS — each requirement should have ONLY ONE interpretation. Ambiguity is a serious defect.

Q5. COCOMO Embedded category is for:

■ PYQ: 2014

- (A) Small, familiar projects
- (B) Medium complexity projects
- ✓ (C) **Real-time, highly constrained projects** ← CORRECT ANSWER
- (D) All types equally

Explanation: Embedded = real-time systems, tight hardware/software constraints. Uses coefficients $a=3.6$, $b=1.20$.

Q6. Function Points (FP) measure software size based on:**■ PYQ: 2018**

- (A) Lines of code
 ✓ (B) **User-visible functionality** ← **CORRECT ANSWER**
 (C) Memory used
 (D) CPU time

Explanation: FP counts: External Inputs + External Outputs + Inquiries + Internal Logical Files + External Interface Files — all user-visible.

Q7. If every requirement in SRS can be checked by a cost-effective process, the SRS is:**■ PYQ: 2019**

- (A) Traceable
 (B) Modifiable
 (C) Complete
 ✓ (D) **Verifiable** ← **CORRECT ANSWER**

Explanation: Verifiable = every requirement can be tested cost-effectively. Non-verifiable: 'system must be user-friendly' — too vague.

Q8. Delphi technique for cost estimation involves:**■ PYQ: 2022**

- (A) One expert giving estimate
 ✓ (B) **Multiple rounds of anonymous expert opinions** ← **CORRECT ANSWER**
 (C) Mathematical formula only
 (D) LOC counting only

Explanation: Delphi = multiple experts estimate anonymously in rounds until consensus — avoids bias from dominant personalities.

SECTION B — 4-MARK SHORT ANSWERS**Q1. Briefly explain the Delphi Cost Estimation Technique.****4 Marks****■ PYQ: 2022****📌 ANSWER:****Definition:**

Delphi Technique is a structured group estimation method where multiple experts provide cost estimates **independently and anonymously** over multiple rounds until consensus is reached.

Process (3 Rounds):

1. **Round 1:** Each expert submits estimate privately — no group discussion, no influence from others.
2. **Round 2:** Coordinator shares average results anonymously. Experts see others' estimates and revise their own.
3. **Repeat:** Continue rounds until experts mostly agree (consensus reached).

Why it works: No one feels pressure to match the boss or loudest person. **Independent thinking** + **anonymous feedback** leads to accurate estimates.

Advantages: Reduces bias, documents reasoning, good for novel projects with no historical data.

Disadvantages: Time-consuming (2-4 weeks), needs skilled facilitator, requires expert availability.

Q2. What are the characteristics of a good SRS document?

4 Marks

■ PYQ: 2018, 2019, 2020

📌 **ANSWER:**

SRS = System Requirements Specification

SRS is a formal document describing **WHAT the system must do** — it is the official contract between users and developers.

6 Key Characteristics:

1. **Correct:** Every requirement accurately reflects actual user needs.
2. **Unambiguous:** Each requirement has ONLY ONE possible interpretation.
3. **Complete:** ALL functional and non-functional requirements are included. Nothing missing.
4. **Consistent:** No contradictions exist between any two requirements.
5. **Verifiable:** Each requirement can be tested cost-effectively — measurable criteria stated.
6. **Modifiable:** SRS can be updated easily while maintaining consistency.

IEEE 830 Standard Structure: Section 1 (Introduction) → Section 2 (Overall Description) → Section 3 (Specific Requirements)

Q3. Differentiate between LOC and Function Points (FP) as size metrics.

4 Marks

■ PYQ: 2018, 2020

LOC vs FUNCTION POINTS

Comparison Aspect	LOC (Lines of Code)	Function Points (FP)
Measurement Basis	Physical lines of code	User-visible functionality
Language Dependency	Highly dependent on language	Language-independent
Estimation Timing	Post-design (late)	Early requirements stage (early)
Stakeholder Focus	Technical/code perspective	User/business perspective
Counting Components	Executable statements	Inputs+Outputs+Inquiries+Files+Interfaces
Main Advantage	Simple to count	Unbiased, available early
Main Disadvantage	Penalizes good programmers	Complex, needs expertise to calculate

SECTION C — 12-MARK LONG ANSWERS

Q1. Explain the Basic COCOMO Model. What are its three categories? Give formulas and examples.

12 Marks

■ PYQ: 2014

📌 **ANSWER:**

Introduction:

COCOMO (Constructive Cost Model) was developed by Barry Boehm in 1981 based on analysis of 63 actual software projects. It provides **empirically validated mathematical formulas** to estimate effort, duration, and cost of software projects.

Basic COCOMO Formula:

$$\text{Effort (Person-Months)} = a \times (\text{KLOC})^b$$

$$\text{Duration (Months)} = c \times (\text{Effort})^d$$

ANSWER:

Three Categories of Projects:

COCOMO PROJECT CATEGORIES						
Category	a	b	c	d	Characteristics	Example
Organic (Simple)	2.4	1.05	2.5	0.38	Small team, familiar tech, relaxed environment	Payroll system
Semidetached (Medium)	3.0	1.12	2.5	0.35	Medium team, mixed experience, moderate constraints	Database system
Embedded (Complex)	3.6	1.20	2.5	0.32	Real-time, tight hardware/software constraints, strict	Flight control system

ANSWER:

Example Calculation (Semidetached — 50 KLOC):

- **Effort** = $3.0 \times (50)^{1.12} = 3.0 \times 56.7 = 170.1$ person-months
- **Duration** = $2.5 \times (170.1)^{0.35} \approx 2.5 \times 7.2 \approx 18$ months
- **Team Size** $\approx 170.1 \div 18 \approx 9\text{--}10$ developers

Advantages:

1. **Empirically validated** against 63 real projects — not just theoretical.
2. **Consistent and reproducible** — same inputs always give same estimate.
3. **Accommodates different complexity levels** through three categories.
4. **Well-documented** with extensive published literature.

Limitations:

1. **Accuracy depends on KLOC estimation** — garbage in, garbage out.
2. **Less accurate for novel projects** with no historical data.
3. **Static coefficients** may not reflect modern development environments.
4. **Difficult to classify** project into correct category accurately.

Conclusion:

COCOMO provides a **systematic mathematical basis** for software cost estimation. Its accuracy depends on correct project classification and size estimation. For better accuracy, use **Intermediate COCOMO** which adds 15 cost drivers.

UNIT 3 — SOFTWARE DESIGN

Classification | Design Approaches | Function-Oriented Design | DFD | Structure Charts | OOD

SECTION A — MCQ WITH ANSWERS (2 Marks Each)

Q1. The tool used for Structured/Function-Oriented designing is:

■ PYQ: 2018

- (A) Program flowchart
- ✓ (B) **Structure chart** ← CORRECT ANSWER
- (C) Data Flow Diagram
- (D) Module

Explanation: Structure Chart shows module hierarchy and calling relationships — the primary tool for structured/functional design.

Q2. DFD (Data Flow Diagram) shows:

■ PYQ: 2018, 2025

- (A) Only data flows
- (B) Only processes
- (C) Only data stores
- ✓ (D) **All of the above (data flows, processes, and data stores)** ← CORRECT ANSWER

Explanation: DFD shows ALL three: arrows=data flows, circles=processes, open rectangles=data stores, squares=external entities.

Q3. The Context Diagram of a DFD is also known as:

■ PYQ: 2019

- ✓ (A) **Level 0 DFD** ← CORRECT ANSWER
- (B) Level 1 DFD
- (C) Level 2 DFD
- (D) None of the above

Explanation: Context diagram = Level 0 DFD — shows ENTIRE system as ONE single process with all external entities.

Q4. Which box specifies behaviour of a system based on inputs/outputs only?

■ PYQ: 2018

- (A) State box
- (B) Clear box
- ✓ (C) **Black box** ← CORRECT ANSWER
- (D) None of the above

Explanation: Black box: system treated as having UNKNOWN internals — only inputs and resulting outputs are considered by tester.

Q5. Object-Oriented Design primarily uses which approach?

■ PYQ: 2019, 2025

- (A) Top-Down
- ✓ (B) **Bottom-Up** ← CORRECT ANSWER
- (C) Structured programming
- (D) Waterfall approach

Explanation: OOD uses bottom-up: build basic reusable objects first, then combine them to create larger subsystems.

Q6. Which is the basic design principle that allows hiding internal implementation details?**■ PYQ: 2024**

- (A) Inheritance
- (B) Polymorphism
- ✓ (C) **Abstraction** ← **CORRECT ANSWER**
- (D) Encapsulation

Explanation: Abstraction: hide complex implementation details, expose only essential features/interface to user.

Q7. Function-Oriented Design is based on which principle?**■ PYQ: 2017, 2019**

- (A) Organization around objects
- ✓ (B) **Decomposition into functions with DFD** ← **CORRECT ANSWER**
- (C) Bottom-up composition
- (D) Risk analysis

Explanation: FOD views system as set of functions. Uses DFD for analysis and Structure Charts for design — top-down decomposition.

Q8. Which relationship in OOP allows a child class to inherit properties from parent class?**■ PYQ: 2019, 2024**

- (A) Abstraction
- (B) Encapsulation
- (C) Polymorphism
- ✓ (D) **Inheritance** ← **CORRECT ANSWER**

Explanation: Inheritance: child class acquires attributes and methods of parent class. Example: Sparrow inherits from Bird.

SECTION B — 4-MARK SHORT ANSWERS**Q1. Distinguish between DFD and Flowchart.****4 Marks****■ PYQ: 2014****DFD vs FLOWCHART**

Feature	DFD	Flowchart
Purpose	Shows how DATA flows between system processes	Shows sequence of LOGIC steps/control flow
Level	System-level (what system does)	Module/function level (how it does it)
Decision boxes	Not shown — no diamond shapes	Yes — diamonds show Yes/No decisions
Used in	System analysis and logical design phase	Detailed program design and coding phase
Loops/Conditions	Not represented	Fully shown with arrows
Example use	Model order processing system	Show if-else logic for login validation

Q2. Differentiate between Function-Oriented Design (FOD) and Object-Oriented Design (OOD).**4 Marks****■ PYQ: 2018, 2019, 2024****FOD vs OOD**

Aspect	FOD	OOD
Basic Unit	Function / Procedure	Object (Data + Behavior together)

Aspect	FOD	OOD
Approach	Top-Down (decomposition)	Bottom-Up (composition)
Data Handling	Data shared globally across functions	Data encapsulated inside objects (hidden)
Reusability	Limited — functions tied to data structures	High — via Inheritance and Composition
Notation Tool	DFD + Structure Charts	UML (Class Diagram, Sequence Diagram)
Best Suited For	Stable, procedural, computation-heavy systems	Complex, evolving, large systems
Example	Banking transaction processing	Student Management System with classes]

Q3. What do you mean by Balancing a DFD? Illustrate with example.

4 Marks

■ PYQ: 2014

📌 ANSWER:

Definition of Balancing:

Balancing a DFD means ensuring that the data flows entering and leaving a process in a higher-level DFD (Level N) **exactly match** the data flows in its detailed lower-level DFD (Level N+1).

The Balancing Rule:

Inputs and outputs of a process at Level N = Inputs and outputs at Level N+1 for the SAME process.

Example:

■ DIAGRAM — BALANCED DFD EXAMPLE

Level 0 (Context Diagram): [Customer] → "Order" → (ORDER SYSTEM) → "Invoice" → [Customer] Level 1 (Expanded – MUST have same external flows): [Customer] → "Order" → (1.0 Validate Order) → (2.0 Process Order) → (3.0 Generate Invoice) → "Invoice" → [Customer] ✓ **BALANCED**: Level 1 still receives "Order" and produces "Invoice" ✗ **UNBALANCED**: If Level 1 shows "Payment" input that Level 0 doesn't have

Why Important: Unbalanced DFD means data mysteriously appears or disappears — a **design inconsistency error** that must be corrected.

SECTION C — 12-MARK LONG ANSWERS

Q1. Explain DFD (Data Flow Diagram) with symbols, levels, and rules. What are its limitations?

12 Marks

■ PYQ: 2014, 2018, 2019, 2025

📌 ANSWER:

Introduction:

A **DFD (Data Flow Diagram)** is a graphical representation showing how data flows through a system — what processes transform the data, where it is stored, and what enters/exits the system. It is a key tool in **Function-Oriented Design** and Structured Analysis.

Four DFD Symbols (Gane-Sarson Notation):

1. **Process (Rounded Rectangle/Circle):** A function that transforms input data to output. Named with a verb phrase. Example: "Validate Order", "Calculate Tax".
2. **Data Flow (Arrow with Label):** Shows movement of data between components. Arrow direction = data direction. Example: → "Customer Name", → "Invoice Amount".
3. **Data Store (Open Rectangle — parallel lines):** Where data is stored at rest. Example: [Customer Database], [Order File].
4. **External Entity (Rectangle/Square):** Person or system outside the system boundary — source or destination of data. Example: Customer, Manager, Bank.

Three Levels of DFD:

1. **Level 0 — Context Diagram:** Entire system shown as ONE single process/circle. Shows all external entities and data flows entering/leaving system. Big-picture view only.
2. **Level 1 DFD:** The Level 0 single circle is EXPANDED to show major sub-processes, data stores, and flows between them.
3. **Level 2+ DFD:** Each Level 1 process is further expanded into its detailed sub-processes. Continue until each process is simple enough to code directly.

■ DIAGRAM — DFD LEVELS — LIBRARY MANAGEMENT SYSTEM

LEVEL 0 (Context Diagram): [Student] ■■"Book Request"■■■ (LIBRARY SYSTEM) ■■"Book Issued"■■■
 [Student] [Librarian] ■■"Catalogue Update"■■■ (LIBRARY SYSTEM) ■■"Reports"■■■ [Librarian] LEVEL 1
 (Expanded — 4 processes visible): [Student] ■■"Request"■■■ (1.0 Check Availability) ■■check■■■
 [=Book DB=] ■■"Available" ■■ (2.0 Issue Book) ■■update■■■ [=Issue Records=] ■■"Issue Slip"■■■
 [Student] [Student]■■■"Return"■■■(3.0 Return Book) ■■ (4.0 Calculate Fine)■■■"Fine"■■■[Student]

DFD Rules (Must Follow):

1. Every process must have **at least one input AND one output** — no black holes, no miracles.
2. Data **cannot flow directly** between two data stores — must go through a process.
3. Data **cannot flow directly** between two external entities — must go through a process.
4. Each process must be **labeled with a clear verb phrase**: "Validate", "Calculate", "Generate".
5. **DFD must be balanced** — data flows into/out of a process must match across levels.

Limitations of DFD:

1. **No control flow** — cannot show loops, if-else decisions, or timing of operations.
2. **Cannot model real-time systems** well — no events, interrupts, or concurrency.
3. **No error handling** — what happens when a process fails is not shown.
4. **Becomes complex** for large systems — hundreds of processes and flows get cluttered.
5. **Static view** — does not show how data changes state over time.

Conclusion:

DFD is an excellent **logical analysis tool** for understanding and modeling data flow in a system. It must be leveled (Level 0 → Level 1 → Level 2) and **balanced at each level** for consistency. For control flow logic, use **flowcharts** or **state diagrams** alongside DFDs.

UNIT 4 — CODING AND TESTING + SOFTWARE RELIABILITY

Unit Testing | White-Box Testing | Black-Box Testing | Debugging | System Testing | Musa's Model

SECTION A — MCQ WITH ANSWERS (2 Marks Each)

Q1. All modules of system are integrated and tested as complete system in the case of:

■ PYQ: 2014

- (A) Bottom-up testing
- ✓ (B) **Big-bang testing** ← CORRECT ANSWER
- (C) Top-down testing
- (D) Sandwich testing

Explanation: Big-bang integration testing: ALL modules integrated at once and tested together as a complete system — high risk.

Q2. The main purpose of Integration Testing is to find:

■ PYQ: 2014

- (A) Design errors
- (B) Analysis errors
- (C) Procedure errors
- ✓ (D) **Interface errors** ← CORRECT ANSWER

Explanation: Integration testing specifically finds INTERFACE errors — problems when modules communicate or exchange data with each other.

Q3. Black-box testing tests software based on:

■ PYQ: 2014, 2019

- (A) Internal code structure
- ✓ (B) **External inputs and expected outputs** ← CORRECT ANSWER
- (C) Database design
- (D) System architecture

Explanation: Black-box: tester treats system as unknown internals. Only tests inputs → expected outputs. No code knowledge needed.

Q4. White-box testing is also known as:

■ PYQ: 2017

- (A) Functional testing
- ✓ (B) **Structural testing (Glass-box testing)** ← CORRECT ANSWER
- (C) System testing
- (D) Acceptance testing

Explanation: White-box = Glass-box = Clear-box = Structural testing. Tester has FULL access to source code and internals.

Q5. Boundary Value Analysis focuses on testing:

■ PYQ: 2022

- (A) Random inputs
- ✓ (B) **Values at the boundaries of input ranges** ← CORRECT ANSWER
- (C) Middle values only
- (D) Maximum values only

Explanation: BVA: errors occur most often at BOUNDARIES of valid ranges. Tests: min, min+1, max-1, max values.

Q6. Musa's Basic Model is used for:**■ PYQ: 2014**

- (A) Cost estimation
 ✓ (B) **Software reliability prediction based on execution time** ← **CORRECT ANSWER**
 (C) DFD creation
 (D) Project scheduling

Explanation: Musa's Basic Execution Time Model (1975): predicts failure RATE over time based on CPU execution time, not calendar time.

Q7. The process of fixing bugs after detection is called:**■ PYQ: 2024, 2025**

- (A) Testing
 ✓ (B) **Debugging** ← **CORRECT ANSWER**
 (C) Verification
 (D) Validation

Explanation: Debugging = identify + locate + fix errors. Testing FINDS bugs. Debugging FIXES them. Two different activities.

Q8. Which testing technique divides inputs into groups where each group should behave the same?**■ PYQ: 2022**

- (A) Boundary Value Analysis
 ✓ (B) **Equivalence Partitioning** ← **CORRECT ANSWER**
 (C) White-box coverage
 (D) Path testing

Explanation: Equivalence Partitioning: divide inputs into partitions — test ONE representative value per partition. Reduces test cases.

SECTION B — 4-MARK SHORT ANSWERS**Q1. What is the difference between Black-box and White-box Testing?****4 Marks****■ PYQ: 2014, 2019****BLACK-BOX vs WHITE-BOX TESTING**

Feature	Black-Box Testing	White-Box Testing
Definition	Tests functionality WITHOUT knowing internal code	Tests internal CODE STRUCTURE and logic
Code Knowledge	NOT required — treats system as black box	REQUIRED — full access to source code
Performed By	Software testers / QA team	Software developers
What It Tests	Inputs vs expected outputs (requirements)	Logic paths, code branches, statements
Also Called	Functional testing	Structural testing / Glass-box testing
Techniques	Equivalence Partitioning, Boundary Value Analysis	Statement, Branch, Path coverage
Level Used	System testing, Acceptance testing	Unit testing, Integration testing
Time	Less time-consuming	Most time-consuming

Q2. What is Regression Testing? When is it done? Why necessary?**4 Marks****■ PYQ: 2014**

 ANSWER:**Definition:**

Regression Testing is the process of re-testing previously working software after making changes (bug fixes, new features, refactoring) to ensure existing functionality still works correctly.

When is it Done?

1. After **bug fixes** — ensure the fix didn't break other parts.
2. After **adding new features** — ensure new code doesn't affect old features.
3. After **code refactoring** — code restructured but should behave the same.
4. After **environment changes** — OS upgrade, database version change, new hardware.

Why Necessary?

A change to fix one bug can **accidentally break other working features** — called a "regression." Regression testing catches these unintended side effects before they reach users.

How Performed: Run SAME test cases as before the change + NEW test cases for the change. Often **automated** using tools like Selenium, JUnit, TestNG.

Q3. What is Musa's Basic Model for Software Reliability? Explain its formula.**4 Marks**■ **PYQ: 2014** **ANSWER:****Definition:**

Musa's Basic Execution Time Model (1975) is a software reliability model that predicts the **failure intensity** (failure rate) as a function of **CPU execution time** — not calendar time.

Core Formula:

$$\lambda(\tau) = f \times K \times [N - \mu(\tau)]$$

Where:

1. $\lambda(\tau)$ = Failure intensity (failure rate) at execution time τ
2. f = Linear execution frequency of the program
3. K = Fault exposure ratio (proportion of faults that cause failures)
4. N = Total number of faults at the start of testing
5. $\mu(\tau)$ = Expected number of failures experienced by execution time τ

Key Insight: As faults are detected and removed, **failure rate DECREASES** over time. The model assumes failures follow a **Poisson process** with exponential distribution.

SECTION C — 12-MARK LONG ANSWERS**Q1. Explain White-box and Black-box testing in detail with all techniques. Advantages and disadvantages.****12 Marks**■ **PYQ: 2014, 2019, 2022** **ANSWER:**

Introduction:

Software testing is a critical quality assurance activity. The two fundamental approaches — **White-box (Structural)** and **Black-box (Functional)** testing — examine software from completely different perspectives. Modern testing uses BOTH for comprehensive coverage.

PART 1 — WHITE-BOX TESTING:

Definition: **White-box testing** (also Glass-box, Clear-box, Structural testing) is a method where the tester has **complete knowledge of and access to the source code**, internal structure, and design of the software.

White-Box Testing Techniques:

1. **Statement Coverage:** Ensure every executable statement in the source code is executed at least once. Formula: $\text{Coverage} = (\text{Statements executed} \div \text{Total statements}) \times 100\%$ Simplest but weakest form — does not test all outcomes of conditions.
2. **Branch Coverage (Decision Coverage):** Ensure every possible outcome of EVERY decision point is tested (both TRUE and FALSE branches of if-else, while, for). Stronger than statement coverage — tests both sides of each decision.
3. **Path Coverage:** Execute EVERY possible execution path through the code. Most thorough — guarantees all logic is tested. Most expensive — number of paths can be exponential.

White-Box Advantages: Thorough code coverage | Finds hidden logic errors | Measures code quality | Enables removing dead code

White-Box Disadvantages: Time-consuming and expensive | Requires programming expertise | Cannot find missing requirements

PART 2 — BLACK-BOX TESTING:

Definition: **Black-box testing** (Functional testing) is a method where the tester treats the system as a "black box" — **NO knowledge of internal code**. Only inputs and expected outputs are considered.

Black-Box Testing Techniques:

1. **Equivalence Partitioning:** Divide ALL possible inputs into partitions (groups) of valid and invalid values. Assumption: if one value in a partition works, ALL values in that partition behave similarly. Example — Age input (valid: 1-120): Partition 1: <1 (invalid) | Partition 2: 1-120 (valid) | Partition 3: >120 (invalid) Test one value from EACH partition: test -5, test 50, test 200.
2. **Boundary Value Analysis (BVA):** Test values AT THE BOUNDARIES of input ranges — errors occur most often at edges. Example — Same Age input (1-120): Test values: 0, 1, 2, 119, 120, 121 (min-1, min, min+1, max-1, max, max+1)

Black-Box Advantages: No code knowledge needed | Tests from user perspective | Finds requirement gaps | Can be done by non-programmers

Black-Box Disadvantages: Cannot test all internal paths | May miss logic errors | Can be redundant without planning

Summary Comparison:**WHITE-BOX vs BLACK-BOX COMPARISON**

Feature	White-Box	Black-Box
Knowledge of Code	Required — full access	Not required — black box
Performed By	Developers	Testers / QA
Tests	Internal structure, logic paths	External functionality, requirements
Techniques	Statement, Branch, Path coverage	Equivalence Partitioning, BVA
Level Used	Unit, Integration testing	System, Acceptance testing
Finds	Logic errors, dead code, uncovered paths	Missing features, wrong outputs

 ANSWER:

Conclusion:

White-box and Black-box testing are **complementary, not competing** approaches. White-box ensures code quality and coverage; Black-box validates user requirements. Best practice: use **White-box for unit/integration testing** and **Black-box for system/acceptance testing** to achieve comprehensive software quality.

UNIT 5 — SOFTWARE QUALITY ASSURANCE + SOFTWARE MAINTENANCE

ISO 9000 | SEI CMM (5 Levels) | Maintenance Types | Process Models | Reverse Engineering | Maintenance Costs

SECTION A — MCQ WITH ANSWERS (2 Marks Each)

Q1. The primary aim of Software Engineering is to provide:

■ PYQ: 2019

- (A) Reliable software only
- (B) Software meeting requirements only
- (C) Cost-effective software only

✓ (D) All of the above ← CORRECT ANSWER

Explanation: SE aims for ALL three combined: reliable + meets requirements + cost-effective. No single goal alone is sufficient.

Q2. ISO 9000 is:

■ PYQ: 2024, 2025

- (A) A programming language standard
- ✓ (B) An international quality management system standard ← CORRECT ANSWER
- (C) A database standard
- (D) A network protocol standard

Explanation: ISO 9000 = International Standards Organization quality management standard. Applies to any industry, ensures consistent quality.

Q3. SEI CMM has how many maturity levels?

■ PYQ: 2018

- (A) 3 levels
- (B) 4 levels
- ✓ (C) 5 levels ← CORRECT ANSWER
- (D) 6 levels

Explanation: SEI CMM has exactly 5 levels: 1-Initial, 2-Repeatabeable, 3-Defined, 4-Managed, 5-Optimizing.

Q4. Change made to the system to reduce future system failure chance is called:

■ PYQ: 2014

- (A) Corrective maintenance
- (B) Adaptive maintenance
- (C) Perfective maintenance

✓ (D) Preventive maintenance ← CORRECT ANSWER

Explanation: Preventive maintenance: proactive changes to PREVENT future problems — improving reliability before failures happen.

Q5. The process of fixing defects in a released software product is called:

■ PYQ: 2025

- (A) Adaptive maintenance
- (B) Perfective maintenance

✓ (C) Corrective maintenance ← CORRECT ANSWER

- (D) Preventive maintenance

Explanation: Corrective maintenance: reactive — fixing bugs and defects AFTER they are discovered in released software.

Q6. Which CMM level has processes documented and standardized organization-wide?

■ PYQ: 2018

- (A) Level 2 Repeatable
 ✓ (B) **Level 3 Defined** ← CORRECT ANSWER
 (C) Level 4 Managed
 (D) Level 5 Optimizing

Explanation: CMM Level 3 (Defined): Processes are documented, standardized, and integrated across the entire organization.

Q7. What is Reverse Engineering in Software Maintenance?

■ PYQ: 2024, 2025

- (A) Creating software from scratch
 ✓ (B) **Recreating design specs from existing code** ← CORRECT ANSWER
 (C) Testing software backwards
 (D) Running software in reverse

Explanation: Reverse Engineering: analyze existing code to recreate design documentation when original docs are missing/lost.

Q8. The highest percentage of maintenance effort is spent on:

■ PYQ: 2022

- (A) Corrective maintenance
 (B) Adaptive maintenance
 ✓ (C) **Perfective maintenance (enhancements)** ← CORRECT ANSWER
 (D) Preventive maintenance

Explanation: Perfective maintenance (adding new features/enhancements) accounts for ~50% of all maintenance effort.

SECTION B — 4-MARK SHORT ANSWERS**Q1. What are the 4 types of Software Maintenance? Give examples.****4 Marks**

■ PYQ: 2014, 2020, 2022

📌 ANSWER:

Software Maintenance = All activities to modify software after delivery. Accounts for 60-80% of total software cost.

4 Types:

1. **Corrective Maintenance (~21%):** Fix defects/bugs discovered after release. Example: App crashes when user uploads large file → fix memory handling code.
2. **Adaptive Maintenance (~25%):** Modify software to work in a changed environment. Example: Update software for new Android version | Adjust GST calculation for new tax rate.
3. **Perfective Maintenance (~50%):** Add new features or improve performance based on user requests. Example: Add dark mode | Improve search response time | Add mobile app version.
4. **Preventive Maintenance (~5%):** Changes to PREVENT future failures — improve structure, add better error handling. Example: Refactor messy legacy code | Add comprehensive logging before failure occurs.

Q2. Differentiate between ISO 9000 and SEI CMM with a comparison table.**4 Marks**

■ PYQ: 2018, 2024, 2025

ISO 9000 vs SEI CMM

Feature	ISO 9000	SEI CMM
Full Form	International Standards Organization 9000	Capability Maturity Model
Purpose	Quality Management System — document & follow processes	Assess & improve software process maturity
Structure	Set of standards (ISO 9001 for production)	5 graduated maturity levels (1 to 5)
Applicability	Any industry (manufacturing, services, software)	Specifically for software organizations
Certification	Yes — third-party audit, certificate valid 3 yrs	No certification — assessment-based appraisal
Outcome	Certified (or not) — binary result	Maturity Level rating (1 to 5)
Goal	Ensure processes are documented and consistently followed	Continuously improve process capability
Key Principle	"Say what you do; Do what you say; Prove it"	Process improvement is gradual and staged

Q3. What is the significance of Reverse Engineering in Software Maintenance?

4 Marks

■ PYQ: 2024, 2025

📌 **ANSWER:**

Definition:

Reverse Engineering is the process of analyzing existing software to recreate design documents, specifications, or higher-level abstract representations when **original documentation is unavailable or missing**.

Process of Reverse Engineering:

Source Code → Program Understanding → Design Recovery → Requirements Recovery → Documentation

Significance/Importance:

1. **Legacy System Maintenance:** Old systems with no documentation can be understood and maintained.
2. **System Migration:** Helps move old systems to new platforms by understanding current design.
3. **Security Analysis:** Analyze malware or third-party software for security vulnerabilities.
4. **Component Reuse:** Identify reusable components in existing code for new projects.
5. **Documentation Recovery:** Create missing/lost technical documentation for old systems.

SECTION C — 12-MARK LONG ANSWERS

Q1. Explain ISO 9000 and SEI CMM in detail. What are the 5 CMM levels? Compare them comprehensively.

12 Marks

■ PYQ: 2018, 2022, 2024, 2025

📌 **ANSWER:**

Introduction:

Software quality assurance requires systematic processes and standards. **ISO 9000** and **SEI CMM** are the two most important quality frameworks used globally to ensure and continuously improve software quality.

PART 1 — ISO 9000:

Definition: **ISO 9000** is a family of international standards published by the International Organization for Standardization that defines requirements for a **Quality Management System (QMS)**. It ensures organizations consistently deliver products/services that meet customer requirements.

Key ISO Standards:

1. **ISO 9001:** Most important — sets requirements for a QMS in production, development, installation, and servicing.
2. **ISO 9000:** Fundamentals and vocabulary of quality management systems.
3. **ISO 9004:** Guidelines for performance improvement beyond basic requirements.

ISO 9000 Key Principles:

1. **Customer focus** — primary aim is to meet customer requirements.
2. **Process approach** — activities managed as processes with defined inputs, outputs.
3. **Continual improvement** — ongoing improvement of QMS performance.
4. **Evidence-based decision making** — decisions based on analysis of data.
5. Core Principle: "Say what you do; Do what you say; Prove it."

PART 2 — SEI CMM (Capability Maturity Model):

Definition: **SEI CMM** is a framework developed by Carnegie Mellon's Software Engineering Institute for assessing and improving the maturity of software development processes in an organization. Maturity ranges from chaotic (Level 1) to optimized (Level 5).

Five CMM Maturity Levels:

SEI CMM — 5 MATURITY LEVELS			
Level	Name	Characteristics	Process State
Level 1	Initial	Ad-hoc, chaotic processes. Success depends on individuals, not process.	Unpredictable
Level 2	Repeatable	Basic project management. Requirements managed. Past successes can be repeated.	Sometimes predictable
Level 3	Defined	Processes documented, standardized, and integrated ACROSS the organization.	Defined and consistent
Level 4	Managed	Quantitative control of processes using statistical data and metrics.	Measured and controlled
Level 5	Optimizing	Continuous process improvement based on quantitative feedback. Defect prevention.	Continuously improving

ANSWER:**ISO 9000 vs SEI CMM — Key Differences:**

1. **Focus:** ISO focuses on documenting and FOLLOWING processes; CMM focuses on IMPROVING process maturity over time.
2. **Result:** ISO gives binary certification (certified/not certified); CMM gives 1-5 maturity rating.
3. **Industry:** ISO applies to ANY industry; CMM is specifically for SOFTWARE organizations.
4. **Approach:** ISO is standards-based compliance; CMM is improvement-oriented journey.
5. **Both Together:** Many organizations pursue both — ISO for customer certification + CMM for internal process improvement.

Conclusion:

ISO 9000 ensures processes are consistently followed and documented, while SEI CMM provides a roadmap for **continuously improving** process capability. Together they form a comprehensive quality assurance framework. Most mature software organizations aim for **ISO certification + CMM Level 3 or above**.

Q2.

Explain Software Maintenance in detail. Types, Process Models, Reverse Engineering, and Estimation of Maintenance Costs.

12 Marks

■ PYQ: 2018, 2019, 2020, 2022, 2024, 2025

👉 **ANSWER:**

Introduction:

Software Maintenance is the most expensive and longest phase of the software lifecycle, consuming **60-80% of total software cost**. It involves ALL modifications made to software AFTER delivery to correct faults, improve performance, or adapt to changing environments.

Definition (IEEE 1219):

"The process of modifying a software system or component after delivery to **correct faults, improve performance, adapt to changed environment**, or add new features."

4 Types of Software Maintenance:

1. **Corrective Maintenance (~21%):** Fix defects/bugs found after release. REACTIVE. Example: Mobile app crashes on login — investigate and fix authentication bug.
2. **Adaptive Maintenance (~25%):** Modify software for changed environment. REACTIVE. Example: Update payroll system for new income tax slabs | Migrate from MySQL to PostgreSQL.
3. **Perfective Maintenance (~50%):** Add new features or improve performance. Can be planned. Example: Add voice search feature | Optimize slow database queries | Add multilingual support.
4. **Preventive Maintenance (~5%):** Proactive changes to prevent future failures. PROACTIVE. Example: Refactor spaghetti code | Add comprehensive error logging | Update SSL certificates.

Maintenance Process Models:

1. **Quick Fix Model:** Apply minimum fix immediately without full analysis. Fast but creates technical debt. Risky — fix can introduce new bugs.
2. **Iterative Enhancement Model:** Systematic approach: understand current system → identify change → design fix → implement → test → deliver. Best practice.
3. **Full Reuse Model:** Identify reusable components in existing code; redesign using existing assets. Promotes code reuse.

Reverse Engineering in Maintenance:

1. When original **documentation is missing/lost** — analyze code to recover design.
2. Process: Source Code → Program Understanding → Design Recovery → Requirements Recovery.
3. Used for **legacy systems** that must be maintained but have no documentation.

Estimation of Maintenance Costs:

1. **Why difficult:** Maintenance requests are unpredictable in timing, nature, and complexity.
2. **Rule of thumb:** Annual maintenance $\approx$ 15-20% of original development cost per year.
3. **Factors increasing cost:** Poor code quality | Missing documentation | Staff turnover | Old/legacy systems | Frequent requirement changes.
4. **Estimation approaches:** (a) Cost per change request — estimate effort per bug fix or enhancement (b) Function point approach — measure size of changes in FP, apply productivity rates (c) Lehman's Laws — software must be continuously adapted or it becomes less useful (and maintenance cost grows over time).

Conclusion:

Software maintenance is **inevitable and continuous**. Planning for maintainability from the design phase — through good documentation, modular design, clear coding standards, and proper testing — significantly reduces long-term maintenance costs and effort. **Preventive and perfective** maintenance, though often overlooked, are the best long-term investment in software health.

★ MOST REPEATED QUESTIONS — PYQ FREQUENCY ANALYSIS			
TOPIC	UNIT	TIMES REPEATED	PYQ YEARS
Waterfall Model (phases, adv, disadv)	U1	★ 8×	2014,2017,2018,2019,2020,2022,2024,2025
Software Maintenance (types, models, costs)	U5	★ 8×	2014,2018,2019,2020,2022,2024,2025
ISO 9000 vs SEI CMM	U5	★ 7×	2018,2019,2022,2024,2025
SRS Document (characteristics, structure)	U2	★ 6×	2018,2019,2020,2022,2024,2025
Black-Box vs White-Box Testing	U4	★ 6×	2014,2017,2019,2022,2024,2025
Spiral Model (4 quadrants, risk)	U1	★ 5×	2014,2017,2019,2020,2022
Software Testing Levels	U4	★ 5×	2014,2017,2022,2024,2025
DFD — Symbols + Levels + Balancing	U3	★ 5×	2014,2018,2019,2020,2025
COCOMO Model (formula, categories)	U2	★ 5×	2014,2018,2019,2025
Prototype Model (types, benefits)	U1	★ 4×	2014,2017,2019,2022
Functional vs Non-Functional Requirements	U2	★ 4×	2018,2024,2025
Software Metrics (LOC, FP)	U2	★ 4×	2014,2018,2020,2022
Object-Oriented Design (principles, OOP)	U3	★ 4×	2018,2019,2024,2025
Cohesion & Coupling	U3	★ 3×	2014,2019,2022
Reverse Engineering	U5	★ 3×	2019,2024,2025

■ LAST-MINUTE CHEAT SHEET — EXAM MORNING QUICK READ			
IMPORTANT FULL FORMS			
ABBREVIATION	FULL FORM	ABBREVIATION	FULL FORM
SE	Software Engineering	SDLC	Software Development Life Cycle
LOC	Lines of Code	FP	Function Points
COCOMO	Constructive Cost Model	SLIM	Software Lifecycle Model (Putnam)
SRS	Software Requirements Specification	DFD	Data Flow Diagram
OOD	Object-Oriented Design	FOD	Function-Oriented Design
SQA	Software Quality Assurance	CMM	Capability Maturity Model
IEEE	Institute of Electrical & Electronics Engineers	QMS	Quality Management System
CASE	Computer-Aided Software Engineering	UAT	User Acceptance Testing
BVA	Boundary Value Analysis	RAD	Rapid Application Development

KEY FORMULAS TO REMEMBER:

FORMULA NAME	FORMULA
Basic COCOMO	Effort = $a \times (KLOC)^b$ Duration = $c \times (Effort)^d$
Musa's Reliability Model	$\lambda(\tau) = f \times K \times [N - \mu(\tau)]$
Defect Removal Efficiency	DRE = (Defects found before delivery ÷ Total defects) × 100%
Effort Variance	Effort Variance = Actual Effort – Estimated Effort
Function Points	FP = Raw FP × Technical Complexity Factor
COCOMO Organic	Effort = $2.4 \times (KLOC)^{1.05}$ a=2.4, b=1.05
COCOMO Semidetached	Effort = $3.0 \times (KLOC)^{1.12}$ a=3.0, b=1.12
COCOMO Embedded	Effort = $3.6 \times (KLOC)^{1.20}$ a=3.6, b=1.20

MEMORY TRICKS:

TOPIC	MEMORY TRICK
Waterfall phases	Requirements → Design → Implementation → Testing → Deployment → Maintenance "Really Dumb Idiots Try Doing More" (R-D-I-T-D-M)
Maintenance types	CAPP = Corrective Adaptive Perfective Preventive
CMM Levels 1→5	I Really Do Master Optimizing = Initial Repeatable Defined Managed Optimizing
DFD symbols	APES = Arrow(flow) Process(circle) External entity(rectangle) Store(open box)
COCOMO categories	OSE = Organic Semidetached Embedded (simple → complex, a goes 2.4→3.0→3.6)
SRS characteristics	Can Unambiguous Correct Come Verifiably Modifiable = C-U-C-C-V-M
FOD vs OOD	FOD = Functions first (Top-down) OOD = Objects first (Bottom-up)
Black vs White box	Black = Blind testing White = See-through testing (code visible)

COMMON EXAM MISTAKES — AVOID THESE:

- ✗ Mistake 1:** Confusing Waterfall with Spiral — Waterfall is linear-sequential; Spiral is iterative with risk analysis.
- ✗ Mistake 2:** Confusing Corrective with Adaptive maintenance — Corrective = fix bugs; Adaptive = change for new environment.
- ✗ Mistake 3:** Saying LOC is language-independent — LOC is LANGUAGE DEPENDENT. FP is language-independent.
- ✗ Mistake 4:** Saying Black-box requires code knowledge — Black-box requires NO code knowledge. White-box requires code knowledge.
- ✗ Mistake 5:** Saying Prototype model has NO disadvantages — it DOES have disadvantages: scope creep, budget overruns, poor code structure.
- ✗ Mistake 6:** Writing COCOMO as "Computer Cost Model" — it is CONSTRUCTIVE Cost Model.
- ✗ Mistake 7:** Confusing Integration Testing with System Testing — Integration finds INTERFACE errors; System testing validates ALL requirements.
- ✗ Mistake 8:** Missing the definition before explaining — always start with definition, then go to explanation.